Mid Term Examination Winter 2020

Branch: **5th Sem, B.Tech. CSE** Course: **Operating Systems (CSL309)**
Duration**: 60 minutes** Max. Marks: **25**

Note: Attempt subparts of a question together. State clearly any assumptions and reasoning for the answers

Q1. OS requires certain support from the H/W in order to effectively implement "Limited Direct Execution" mechanism for CPU and memory virtualization. Briefly state any two distinct support/aides required from the H/W and its purpose/reason for each of the following cases: [3 x 2 = 6 marks]

(a) Specific to CPU virtualization mechanism
(b) Specific to Memory virtualization mechanism
(c) Common to both CPU and Memory virtualization mechanism

Q2. Briefly state four distinct reasons (or advantages) why modern OS support/implement memory virtualization. [2 marks]

Q3. Consider a multilevel feedback queue scheduler with three queues, numbered Q0 to Q2 (Q0 being the highest priority queue) and time quantum of 4 ms, 8 ms and 16ms respectively. If a process arrives in a higher priority queue, it will preempt any process in lower queue. Suppose that the processes arrive for executing as per following workload table (numbers are in milliseconds). Assume no I/O and no priority boosting. Let N (in the table) be the last digit of your enrollment no % 4. For ex. If your roll no is BT18CSE045 then N=1(5%4).

Process ID	Arrival Time	CPU Time
P0	0	10 + N
P1	8 + N	05 + N
P2	12 + N	10 - N
P3	15 + N	08 - N

(a) Draw the scheduling of processes with time on X axis and different Queue's on Y axis (as given in book and discussed in class for MLFQ scheduling) and compute following metrics: (b) The average turnaround time (c) The average response time and (d) The average waiting time [2+1+1+1=5 marks]

Q4. One day Govind had an inspiration. He observes that most programs have majority of their code/data at the beginning of the address space. For his homegrown custom OS, Govind decides that he is going to implement paging mechanism using a two-level page table structure with the following twist: The first half of the page table entries in the first level page table (Page Directory) directly maps physical pages, and the second half of the entries map to the second level page tables as normal. Call the first half entries as fast, and the second half as normal. For the following questions, assume that the page size is 2 x (1+N) KB, size of PTE is 4 bytes and that all the page tables fit into a single page. Where N is same as described in Q2 above (i.e the last digit of your enrollment no % 4). [1 x 5=5 marks]

(a) How many virtual pages are fast pages?
(b) How many virtual pages are normal pages?

(c) What is the maximum size of an address space in bytes (use exponential notation for convenience, e.g., 2^3)?
If TLB fetch time is T (with hit ratio as H) and Memory access time is M, what is the time taken for a memory reference whose entry is in:
(d) fast page (e) normal page?

Q5. A system has given specifications: its virtual address is 32 bits long, its page size is 4KB, its physical memory has 1024 page frames and its TLB can store 32 page entries. In this problem, we will try to understand what happens while running some test code. Before running the test code, though, we first run the following *initialization code*.

```
int k =0;
void *store = malloc(NUM_PAGES * PAGE_SIZE);
void *ptr = store;
while(k++ < NUM_PAGES) {
       (int *) *ptr = 0; // initialize first value on each page
       ptr += PAGE_SIZE;
}
```

The initialization code allocates some amount of memory and sets the first integer of each page to 0. Assuming malloc() returns page-aligned memory in this example. After running the initialization code, we run the following *test code*:

```
ptr = store;
k=0;
while(k++ < NUM_PAGES) {
       int x = (int *) *ptr; // load value pointed to by ptr
       ptr += PAGE_SIZE;
}
```

We are only interested in memory accesses to the malloc'd region through ptr and ignore all other memory accesses (i.e., ignore instruction fetches and stores to `x`). We further assume that memory and TLB start from a clean state before we run the initialization code. Given that LRU replacement policy is used whenever such a policy might be needed by either hardware or software:

Fill the table with no. of TLB hits, misses and page faults that occur during the test code when: *[3 marks]*

	TLB hits	TLB misses	Page Faults
(a) `NUM_PAGES` is 32?			
(b) `NUM_PAGES` is 1024?			
(c) `NUM_PAGES` is 2048?			

If TLB lookup time is T, memory access time is M and a disk access time is D, fill in the blanks with the runtime of the test code when: *[3 marks]*

(d) `NUM_PAGES` is 32?	
(e) `NUM_PAGES` is 1024?	
(f) `NUM_PAGES` is 2048?	

(g) How could you use the test code above to learn about the H/W (size of TLB & Memory) of a new machine of which you know very little about*? [1 marks]*

---------- xxx ----------

Course Outcomes:
At the end of the course the students will be able to:

CO1: Describe the implementation mechanisms, policies and functioning of different aspects of operating system.
CO2: Identify, compare and assess issues related to CPU and memory virtualization, process synchronization and file systems
CO3: Design, implement and analyze solutions to various problems in operating systems.

Mapping of Course Outcomes to Program outcomes:

	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PO12**
CO1	Strong						Medium	Medium				
CO2	Strong	Medium										
CO3		Strong			Strong		Medium		Medium	Weak	Weak	

Question Paper to CO mapping:

Mid Term				End Semester			
Q.No.	CO-1	CO-2	CO-3	Q.No.	CO-1	CO-2	CO-3
1	√						
2	√						
3		√					
4		√					
5		√	√				

Assessment of Attainment of Course Outcomes:

	Mid Term (25)	**End Term (35)**	**Quizzes (10)**	**Assignments (30)**
CO1	35%			
CO2	40%			
CO3	25 %			